

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

ADVANCED COMPUTER ARCHITECTURE

Lab 5

Tom Thys

Contents

1 Introduction **2**

2 Asynchronous Operations **2**
2.1 Synchronous vs. Asynchronous Execution 2
2.2 Reduction Task Implementation 2
2.2.1 Benchmark Strategy 2

3 Performance Analysis **2**
3.1 Discussion 3

4 Conclusion **3**

A Appendix: CUDA Kernel Blocks **4**
A.1 Dynamic Parallelism Kernels 4
A.2 Reduction Kernel 4

B Appendix: Host Stream Implementations **4**

1 Introduction

This report evaluates the use of asynchronous CUDA execution with streams. A reduction benchmark is implemented to compare a CPU baseline with synchronous and asynchronous GPU execution.
By performing four independent reduction operations (sum, product, minimum, and maximum), the experiment highlights how overlapping CPU-side data generation with GPU memory transfers and kernel execution can reduce overall runtime. The results show that asynchronous execution is particularly beneficial when CPU preparation time is comparable to GPU execution time.

2 Asynchronous Operations

2.1 Synchronous vs. Asynchronous Execution

The standard CUDA execution model often involves blocking calls (e.g., `cudaMemcpy`) that force the CPU to wait for the GPU. Asynchronous operations allow the CPU to queue work (memory copies and kernel launches) into **Streams**, enabling the CPU to continue processing unrelated tasks while the GPU works.

2.2 Reduction Task Implementation

The assignment required performing four reduction operations (Sum, Product, Min, Max) on independent arrays. We utilized a single kernel, `reduce_kernel`, which accepts an opcode to determine the operation type.

2.2.1 Benchmark Strategy

- **CPU Baseline:** All operations (array generation and reduction) are performed entirely on the CPU without GPU offloading.
- **Synchronous Approach:** We generated data on the CPU, copied it to the GPU, executed the kernel, and waited for completion using `cudaDeviceSynchronize()` sequentially for each of the four arrays. The CPU effectively busy-waits during GPU execution.
- **Asynchronous Approach:** We utilized four distinct `cudaStream_t` objects. By using `cudaMemcpyAsync`, we queued the memory transfer and kernel execution for the "Sum" operation, then immediately proceeded to generate data for the "Product" operation on the CPU while the GPU processed the first stream.

3 Performance Analysis

We bench marked the three implementations (CPU, Synchronous GPU, Asynchronous GPU) over varying list sizes (2^1 to 2^{15} items). The results are averaged over 100 runs.

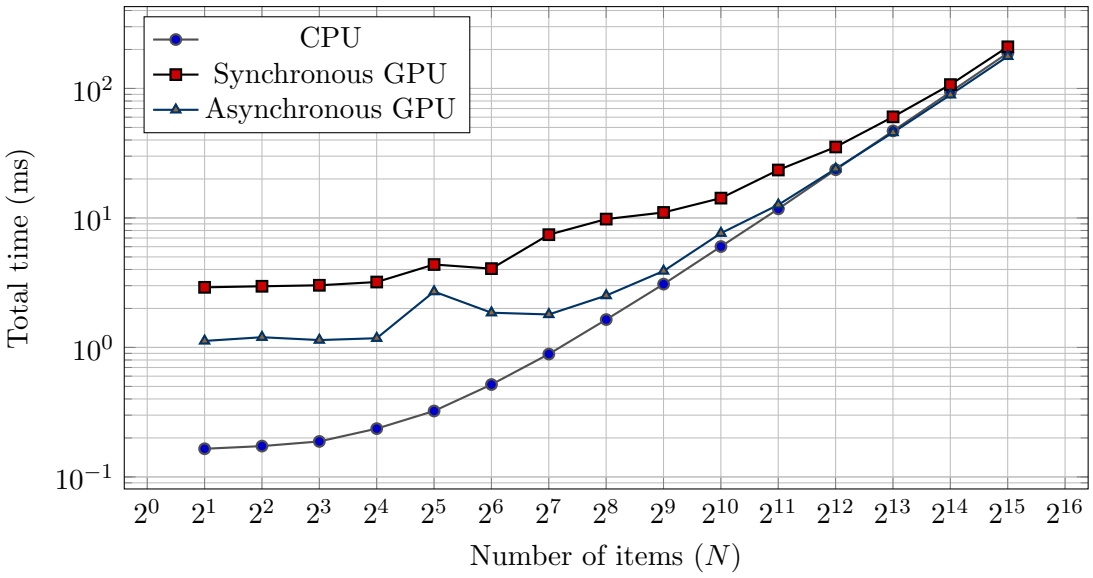


Figure 1: Performance comparison on logarithmic scale (100 runs averaged).

3.1 Discussion

Figure 1 illustrates the total execution time for varying input sizes.

- **Small Data Sizes (< 256 items):** The asynchronous version significantly outperforms the synchronous version. At small sizes, the time required for the CPU to generate random lists is comparable to the execution time of the GPU. In the synchronous model, the CPU is idle (busy-waiting) while the GPU processes the data. In the asynchronous model, the CPU prepares the next list while the GPU is busy with the previous one, effectively hiding the generation latency.
- **Large Data Sizes (> 8192 items):** As the list size grows, random number generation on the CPU becomes very fast relative to the computation and memory transfer overhead. Consequently, the gap between Synchronous and Asynchronous execution narrows. However, the Asynchronous version remains consistently faster because it avoids the overhead of stopping and restarting the GPU pipeline for each of the four operations.

4 Conclusion

The asynchronous implementation demonstrates how overlapping computation (CPU generation of arrays) with communication (PCIe transfer) and GPU execution significantly reduces total runtime. This effect is most pronounced with small-to-medium datasets where the CPU workload (data preparation) can be completely hidden behind GPU execution.

A Appendix: CUDA Kernel Blocks

This appendix contains the specific CUDA kernel implementations used in the assignment.

A.1 Dynamic Parallelism Kernels

The following kernels demonstrate a kernel that launches another kernel directly on the device.

```
__global__ void ChildKernel() {
    printf("  -> Child: block %d, thread %d\n", blockIdx.x, threadIdx.x
        );
}

__global__ void ParentKernel() {
    printf("Parent: block %d, thread %d\n", blockIdx.x, threadIdx.x);

    // only one thread launches child
    if (threadIdx.x == 0 && blockIdx.x == 0) {
        printf("  Launching ChildKernel from device...\n");
        ChildKernel<<<2, 4>>>();  // launch child kernel
        printf("  ChildKernel finished!\n");
    }

    __syncthreads();
}
```

Listing 1: Parent and Child Kernels (child_parents.cu)

A.2 Reduction Kernel

This kernel performs pairwise reduction for Sum, Product, Min, or Max based on the `op` parameter.

```
__global__ void reduce_kernel(int *array, int size, int op) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;

  // simple pairwise reduction pattern
  for (int stride = 1; stride < size; stride *= 2) {
    int index = 2 * stride * idx;
    if (index + stride < size) {
      int a = array[index];
      int b = array[index + stride];
      switch (op) {
        case 0: array[index] = a + b; break;     // sum
        case 1: array[index] = a * b; break;     // prod
        case 2: array[index] = min(a, b); break; // min
        case 3: array[index] = max(a, b); break; // max
      }
    }
    __syncthreads();
  }
}
```

Listing 2: Reduction Kernel (code.cu)

B Appendix: Host Stream Implementations

```
printf("===== SYNCHRONOUS VERSION =====\n");
auto start1 = std::chrono::steady_clock::now();

for (int i = 0; i < 100; i++){
  // 1. Sum Operation
  generateArray(h_sum, N);
  cudaMemcpy(d_sum, h_sum, bytes, cudaMemcpyHostToDevice);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int)>>>(d_sum, N, 0);
  cudaDeviceSynchronize(); // Blocks CPU here

  // 2. Product Operation
  generateArray(h_prod, N);
  cudaMemcpy(d_prod, h_prod, bytes, cudaMemcpyHostToDevice);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int)>>>(d_prod, N, 1);
  cudaDeviceSynchronize(); // Blocks CPU here

  // 3. Min Operation
  generateArray(h_min, N);
  cudaMemcpy(d_min, h_min, bytes, cudaMemcpyHostToDevice);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int)>>>(d_min, N, 2);
  cudaDeviceSynchronize(); // Blocks CPU here

  // 4. Max Operation
  generateArray(h_max, N);
  cudaMemcpy(d_max, h_max, bytes, cudaMemcpyHostToDevice);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int)>>>(d_max, N, 3);
  cudaDeviceSynchronize(); // Blocks CPU here
}

auto end1 = std::chrono::steady_clock::now();
printf("Sync total time: %.3f ms\n",
       std::chrono::duration<double,std::milli>(end1-start1).count());
```

Listing 3: Synchronous Execution Loop

```
printf("\n===== ASYNCHRONOUS VERSION =====\n");

// Create 4 distinct streams
cudaStream_t s1, s2, s3, s4;
cudaStreamCreate(&s1); cudaStreamCreate(&s2);
cudaStreamCreate(&s3); cudaStreamCreate(&s4);

auto start2 = std::chrono::steady_clock::now();

for (int i = 0; i < 100; i++){
  // Launch overlapping work

  // Stream 1: Sum
  generateArray(h_sum, N);
  cudaMemcpyAsync(d_sum, h_sum, bytes, cudaMemcpyHostToDevice, s1);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int), s1>>>(d_sum, N, 0);

  // Stream 2: Prod (Does NOT wait for s1)
  generateArray(h_prod, N);
  cudaMemcpyAsync(d_prod, h_prod, bytes, cudaMemcpyHostToDevice, s2);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int), s2>>>(d_prod, N, 1);

  // Stream 3: Min
  generateArray(h_min, N);
  cudaMemcpyAsync(d_min, h_min, bytes, cudaMemcpyHostToDevice, s3);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int), s3>>>(d_min, N, 2);

  // Stream 4: Max
  generateArray(h_max, N);
  cudaMemcpyAsync(d_max, h_max, bytes, cudaMemcpyHostToDevice, s4);
  reduce_kernel<<<GRID, BLOCK, BLOCK*sizeof(int), s4>>>(d_max, N, 3);
}

// Synchronize all streams at the end
cudaStreamSynchronize(s1);
cudaStreamSynchronize(s2);
cudaStreamSynchronize(s3);
cudaStreamSynchronize(s4);

auto end2 = std::chrono::steady_clock::now();
printf("Async total time: %.3f ms\n",
       std::chrono::duration<double,std::milli>(end2-start2).count());
```

Listing 4: Asynchronous Execution Loop with Streams